

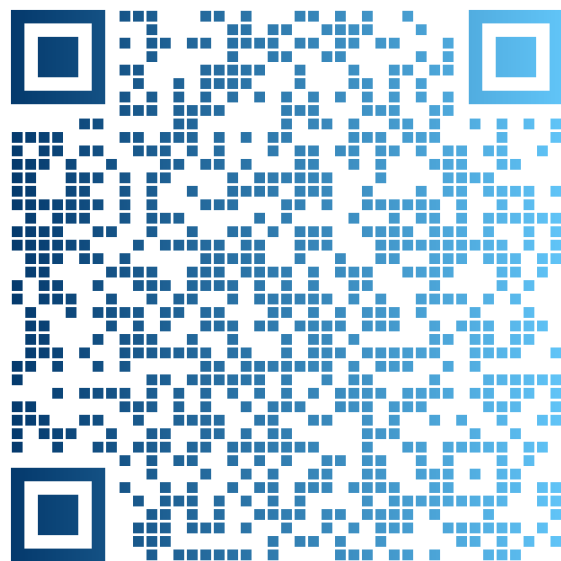


المدرسة العليا للتكنولوجيا
جامعة الحسن الثاني الدار البيضاء
Ecole Supérieure de Technologie
Université Hassan II de Casablanca

Machine Learning

IAOUSSE M'barek

2026



Ce **manuel** a été élaboré pour servir de support de cours principal pour le module *Machine Learning* dispensé dans le cadre de la Licence Professionnelle DBT-DAIA (*Développement d'Applications et Intelligence Artificielle*) en Temps Aménagé à l'École Supérieure de Technologie de Casablanca (ESTC).

L'auteur est M'barek IAOUSSE, Enseignant-Chercheur à l'EST Casablanca, Université Hassan II de Casablanca.



Sauf indications contraires, le contenu de ce manuel électronique est disponible en vertu des termes de la [Licence Creative Commons Attribution 4.0 International](https://creativecommons.org/licenses/by/4.0/).

Vous êtes autorisé à :

Partager – copier, distribuer et communiquer le matériel par tous moyens et sous tous formats.

Adapter – remixer, transformer et créer à partir du matériel pour toute utilisation, y compris commerciale.

Condition : Créditer l'auteur (M'barek IAOUSSE) en cas de réutilisation ou d'adaptation du contenu.

© M'barek IAOUSSE (2026)

Pour citer ce manuel : M'barek IAOUSSE. (2026). *Cours de Machine Learning : Fondements théoriques et applications pratiques en Python*. École Supérieure de Technologie de Casablanca. Sous licence CC BY 4.0.

Préface

Ce manuel a commencé comme notes de cours pour le module *Machine Learning* dispensé dans le cadre de la Licence Professionnelle DBT-DAIA (*Développement d'Applications et Intelligence Artificielle*) en temps aménagé à l'École Supérieure de Technologie de Casablanca (ESTC). À l'origine, la communauté étudiante dans ce programme provient de parcours techniques variés (DUT, BTS, DTS ou diplômes équivalents), ayant déjà acquis un bagage en programmation informatique et en mathématiques appliquées. Ainsi, ces personnes ont déjà appris à programmer en Python et à manipuler des structures de données.

Pendant longtemps, l'enseignement du Machine Learning s'est appuyé sur des ouvrages de référence académiques classiques, tels que *Pattern Recognition and Machine Learning* (Bishop 2006) ou *Hands-On Machine Learning* (Géron 2022). Cependant, la plupart des livres du marché sont soit très théoriques et axés sur les mathématiques pures, soit rédigés exclusivement en anglais avec des études de cas pas toujours adaptées au rythme d'une formation en temps aménagé. De plus, certains aspects comme l'ingénierie des données (*Feature Engineering*), la validation rigoureuse des modèles et la transition progressive vers des environnements de production (VS Code, MLOps) sont devenus indispensables sur le marché du travail. Le présent manuel essaie de répondre à l'ensemble de ces besoins, sous forme de ressource éducative libre (REL).

Pourquoi une ressource éducative libre ?

Aujourd'hui, l'écosystème du Machine Learning repose quasi exclusivement sur des outils et logiciels libres. Dans le cadre de l'enseignement, nous utilisons des plateformes de gestion de code source libre comme GitHub et GitLab, car elles permettent d'évaluer, de valider des modèles et de collaborer dans des communautés d'ingénierie et de science des données. Il est donc naturel d'adopter une démarche analogue pour le contenu de ce manuel, soit une **ressource éducative libre (REL)**.

Pour favoriser la réutilisation et la pérennité du support, nous avons utilisé des logiciels libres de référence :

- Quarto (2023) (le langage source du texte est [Markdown](#)) ;
- Pandoc et LaTeX pour le traitement et la compilation vers les formats Web (HTML), PDF et EPUB ;
- Python et ses bibliothèques scientifiques clés (NumPy, Pandas, Matplotlib, Scikit-Learn) pour les exemples et les travaux pratiques.

Autant que possible, les schémas et figures du manuel sont créés sous forme vectorielle (comme SVG) ou générés directement par du code Python afin de garantir une lisibilité optimale, une indexation par les moteurs de recherche et une bonne accessibilité.

Prérequis

Le contenu de ce manuel est organisé pour les personnes ayant déjà une base et une expérience avec :

- la programmation en langage Python (fonctions, structures de données, bases de programmation orientée objet) ;
- les notions élémentaires de mathématiques pour l'informatique (calcul matriciel de base, dérivées, probabilités et statistiques descriptives) ;
- l'utilisation d'un environnement d'exécution d'analyse de données (Jupyter Notebooks, Google Colab ou VS Code).

La familiarité préalable avec le Machine Learning n'est pas un prérequis : les concepts sont réintroduits progressivement depuis leurs fondements mathématiques jusqu'à leur mise en œuvre logicielle.

Livres et ressources complémentaires

Ce manuel s'inspire des méthodologies d'apprentissage guidé par les données et de modélisation statistique moderne. Pour approfondir certains sujets, des références vers des ouvrages classiques et la documentation officielle sont proposées au fil des chapitres.

Note

Pour les aspects pratiques d'implémentation sous Python, nous renvoyons régulièrement au livre d'A. Géron (2022), noté [GÉRON], ou à la documentation officielle de `Scikit-Learn`.

Mise en garde

Les bibliothèques scientifiques Python évoluant rapidement, veuillez à utiliser un environnement virtuel basé sur Python 3.10 ou supérieur pour garantir la compatibilité des extraits de code présentés dans ce manuel.

Table des matières

Préface	i
Introduction au Machine Learning	1
1. De la Programmation Classique à l'Apprentissage Automatique	1
2. Définition Formelle	1
3. Le Cycle de Vie d'un Projet de Machine Learning	2
4. Fil Conducteur du Module	2
Écosystème et Fondements de l'Apprentissage Automatique	3
1.1 Qu'est-ce que l'apprentissage automatique ?	3
1.2 Panorama de l'écosystème ML	3
1.3 Théorème de projection orthogonale (rappel d'algèbre linéaire)	4
1.4 Auto-évaluation	4
1.5 Vocabulaire et abréviations	5
1.6 Citation	5
1.7 Contenu interactif (<i>disponible uniquement sur le site</i>)	5
Rappel Théorie	5
Application Pratique	5
1.8 Illustration	5
1.9 Résumé du chapitre	6
1.10 Exercices	6
References	7

Introduction au Machine Learning

💡 Objectifs pédagogiques de l'introduction

À la fin de ce chapitre introductif, l'étudiant sera capable de :

- Définir formellement le Machine Learning et le situer dans l'écosystème global de l'IA.
- Comparer le paradigme de programmation classique et l'approche guidée par les données (*Data-Driven*).
- Identifier les grandes étapes du cycle de vie d'un projet de Machine Learning.

1. De la Programmation Classique à l'Apprentissage Automatique

En informatique traditionnelle, l'ingénieur écrit explicitement des règles métiers sous forme de code algorithmique pour traiter des données d'entrée et produire un résultat.

Programmation Classique :

Données d'entrée + Règles (Code) ---> Résultat

Cette approche trouve ses limites lorsque les règles deviennent trop complexes ou impossibles à formaliser manuellement (ex. reconnaissance faciale, détection de fraudes bancaires, traitement du langage naturel).

Le **Machine Learning** inverse ce paradigme : au lieu de programmer les règles, on fournit à l'ordinateur des données d'entrée associées aux résultats observés, et l'algorithme *apprend* automatiquement la fonction mathématique sous-jacente.

Machine Learning :

Données d'entrée + Résultats observés ---> Modèle (Règles induites)

2. Définition Formelle

Selon Tom Mitchell (1997), la définition de référence de l'apprentissage automatique s'énonce ainsi :

Définition 0.1 (Apprentissage Automatique). Un programme informatique est dit **apprendre** d'une expérience E , relativement à une classe de tâches T et à une mesure de performance P , si sa performance sur les tâches de T , mesurée par P , s'améliore avec l'expérience E .

- **La Tâche (T)** : Prédire le prix d'un bien immobilier, classer un e-mail (*Spam* / *Non-Spam*).
- **L'Expérience (E)** : Le jeu de données d'entraînement historique (observations X et étiquettes y).
- **La Performance (P)** : La métrique d'évaluation choisie (Erreur Quadratique Moyenne, Exactitude, Score F_1).

3. Le Cycle de Vie d'un Projet de Machine Learning

Un projet de Machine Learning réussi dépasse le simple choix d'un algorithme. Il s'inscrit dans une démarche méthodologique rigoureuse :

1. **Cadrage du problème** : Traduction du besoin métier en un problème ML (T, E, P) .
2. **Collecte & Nettoyage** : Traitement des valeurs manquantes, gestion des doublons et des aberrations.
3. **Analyse Exploratoire (EDA)** : Visualisation des distributions et analyse des corrélations.
4. **Ingénierie des Caractéristiques (*Feature Engineering*)** : Encodage, normalisation et création de variables.
5. **Modélisation & Entraînement** : Sélection des algorithmes et minimisation de la fonction de coût $\mathcal{L}$.
6. **Évaluation & Validation** : Test du modèle sur des données inédites pour évaluer sa capacité de généralisation.
7. **Industrialisation & Suivi** : Déploiement du modèle en production et surveillance des performances.

4. Fil Conducteur du Module

Tout au long de ce cours, nous explorerons les fondements théoriques et mathématiques de chaque famille d'algorithmes avant de procéder à leur implémentation concrète à travers des ateliers pratiques en Python.

Écosystème et Fondements de l'Apprentissage Automatique

💡 Objectifs du chapitre

À la fin de ce chapitre, l'étudiant sera capable de : - définir l'apprentissage statistique et le situer par rapport à l'intelligence artificielle ; - distinguer les grandes familles d'apprentissage (supervisé, non supervisé, par renforcement) ; - citer les rôles des bibliothèques scientifiques Python de référence (**NumPy**, **Pandas**, **Matplotlib**, **Scikit-Learn**) ; - distinguer un environnement d'exploration (Jupyter/Colab) d'un environnement de production (VS Code).

a remplir : introduction générale du chapitre.

1.1 Qu'est-ce que l'apprentissage automatique ?

a remplir

Le Machine Learning consiste à construire une fonction f qui approxime une relation entre des variables d'entrée X et une sortie y , à partir d'exemples $(X_i, y_i)_{i=1}^n$, plutôt que de la programmer explicitement.

$$\begin{aligned}\hat{y} &= f_{\theta}(X) \\ &= w^T X + b\end{aligned}\tag{0.1}$$

On peut se référer à l'équation 0.1 pour la suite du cours : le problème de l'apprentissage consiste à trouver les paramètres optimaux qui minimisent une fonction de coût $\mathcal{L}$:

$$\hat{\theta} := \operatorname{argmin}_{\theta} \mathcal{L}(f_{\theta}(X), y)$$

1.2 Panorama de l'écosystème ML

a remplir

TABLE 1 – Principales bibliothèques Python utilisées dans ce cours

Bibliothèque	Rôle
NumPy	Calcul matriciel et vectoriel
Pandas	Manipulation de données tabulaires
Matplotlib	Visualisation de données
Scikit-Learn	Modélisation, pipelines, métriques

Bibliothèque	Rôle
TensorFlow	Apprentissage profond

Le tableau 1 résume les outils qui seront utilisés dans les TP.

i Environnements d'exécution

- **Exploration** : Jupyter Notebook, Google Colab — pratiques pour l'analyse rapide et la visualisation.
- **Production** : Visual Studio Code — utilisé à partir du Module 4 pour structurer un projet ML complet (scripts, tests, versionnage).

1.3 Théorème de projection orthogonale (rappel d'algèbre linéaire)

a remplir : lien avec la régression linéaire par moindres carrés (Module 3).

Théorème 0.1 (Théorème de projection orthogonale). *Étant donné $y \in \mathbb{R}^n$ et un sous-espace linéaire $S \subset \mathbb{R}^n$, il existe une unique solution au problème de minimisation*

$$\hat{y} := \operatorname{argmin}_{z \in S} \|y - z\|$$

Le minimiseur $\hat{y}$ est l'unique vecteur de $\mathbb{R}^n$ qui satisfait :

- $\hat{y} \in S$
- $y - \hat{y} \perp S$

*Le vecteur $\hat{y}$ est appelé la **projection orthogonale** de y sur S .*

Preuve. Soit z un autre point quelconque de S . En utilisant le fait que S est un sous-espace linéaire, on peut montrer que :

$$\|y - z\|^2 = \|(y - \hat{y}) + (\hat{y} - z)\|^2 = \|y - \hat{y}\|^2 + \|\hat{y} - z\|^2$$

Donc $\|y - z\| \geq \|y - \hat{y}\|$, ce qui achève la démonstration. □

La démonstration du Théorème 0.1 se trouve ci-dessus.

1.4 Auto-évaluation

- ☒ Comprendre la différence entre exploration et production
- ☐ Maîtriser la manipulation matricielle avec NumPy (voir TP 1)

1.5 Vocabulaire et abréviations

a remplir

C'est pour la licence DAIA que ce support de cours est conçu.

Apprentissage supervisé Famille d'algorithmes appris à partir d'exemples étiquetés (X_i, y_i) .

Apprentissage non supervisé Famille d'algorithmes appris sans étiquette, cherchant une structure dans les données (voir Module 5).

Sur-apprentissage Situation où un modèle apprend le bruit de l'échantillon d'entraînement au détriment de sa capacité de généralisation.

On pourra utiliser le terme **Sur-apprentissage** tout au long du cours pour désigner ce phénomène.

1.6 Citation

Le meilleur modèle n'est pas celui qui mémorise le mieux les données d'entraînement, mais celui qui généralise le mieux à des données inédites.

— Principe fondamental du Machine Learning

Ce principe sera formalisé mathématiquement au Module 3 avec la notion de sur-apprentissage¹.

1.7 Contenu interactif (*disponible uniquement sur le site*)

i Cette section n'apparaît pas dans le PDF

Les onglets et le contenu déroulant ci-dessous sont des fonctionnalités interactives de la version en ligne. La version PDF conserve le texte équivalent, sans l'interactivité.

Rappel Théorie

L'apprentissage automatique repose sur la minimisation d'une fonction de coût $\mathcal{L}$.

Application Pratique

En TP, cette notion sera manipulée directement avec **Scikit-Learn** (voir TP 1).

1.8 Illustration

Voir figure 1.

1. On parle aussi de *surajustement*, par opposition au *sous-apprentissage* (*underfitting*), où le modèle est trop simple pour capturer la structure des données.



FIGURE 1 – Support de cours réalisé dans le cadre de la Licence DBT-DAIA.

1.9 Résumé du chapitre

! À retenir

- L'apprentissage automatique remplace une programmation explicite par une estimation à partir de données.
- Le choix de l'environnement (exploration vs production) dépend de la maturité du projet.
- Les notions de biais/variance et de sur-apprentissage seront approfondies au Module 3.

1.10 Exercices

Exercice 0.1. Citer trois différences entre un notebook Jupyter d'exploration et un projet Python structuré pour la production.

💡 Solution de l'exercice Exercice 0.1

a remplir : corrigé détaillé.

References

Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning*. Springer.

Géron, Aurélien. 2022. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.

Posit Software. 2023. *Quarto Documentation*. <https://quarto.org>.